

1. Managing Manual QA + Automation QA

You will be responsible for both Manual and Automation engineers. How would you organize the work between these roles to ensure:

- **efficient testing of landing pages;** Not everything should be automated. Skilled manual QA is essential for exploratory testing and catching difficult edge cases. However, automation provides the foundation for frequent testing, early detection, PR gates, and rapid cross-browser/cross-platform coverage. By automating repetitive checks and complex data flows, we free up manual QA engineers to focus entirely on high-value edge cases and maintaining comprehensive test coverage.
- **clear ownership and task distribution;** We will use task management systems (like Jira) aligned with the development and design sprint cycles. Every ticket will include specific sub-tasks for both automation and manual release sign-off. These tasks will have independent estimations, clear assignees, and defined dependencies (e.g., manual release checks begin only after automated tests pass).
- **smooth collaboration and communication;** Clear communication is critical for remote teams. I encourage daily check-ins, dedicated chat channels for immediate blocker resolution, and close collaboration with developers and designers to proactively anticipate testing needs. Additionally, I organize cross-training sessions to upskill manual QAs in automation. This minimizes siloed knowledge and mitigates risks related to employee absence or turnover.
- **predictable delivery within sprints?** Predictable delivery relies on accurate sprint planning and estimations. While rare exceptions (like illness or urgent cross-team requirements) occur, consistent delays indicate a planning flaw. I ensure predictability by accurately estimating workloads upfront. If a sprint goal is missed, we treat it as a retrospection opportunity: diagnose the root cause, implement a fix, and iterate.

2. Inheriting an Existing Playwright Project

You join a team where automation already exists. What are your first steps to:

- **understand the current state of the framework and tests;** I start by reviewing CI/CD pipelines (scheduled runs, PR jobs) to identify frequent failures, execution bottlenecks, and skipped tests. Next, I conduct a thorough code review to understand the architecture and validate the accuracy of existing assertions. From this, I create and prioritize a backlog of improvements, focusing on quick wins that deliver the highest immediate value.
- **stabilize the test runs;** I prioritize fixing the most frequent points of failure by diagnosing root causes. To ensure stability, I enforce a strict standard: any fixed or newly introduced test must pass 100 consecutive executions (e.g., using Playwright's `--repeat-each` command) with an accepted flakiness rate of under ~2% before being merged.

- **introduce coding and structural standards;** After evaluating the existing setup, I enforce mandatory CI-level linting and static analysis. I also introduce standardized PR templates, leverage automated/AI reviewers where applicable, and ensure all coding standards are documented centrally (e.g., Confluence or Notion).
- **prepare the project for long-term maintainability?** I focus on modularity and adaptability in the framework architecture. Improvements are prioritized, estimated, and implemented incrementally. While agentic solutions can assist with maintenance and code generation, their outputs must always be rigorously verified by the team.

3. Working with Designers

Designers frequently update landing pages. How would you:

- **ensure QA and Design stay aligned on quality expectations;** I establish a standard workflow where QA is notified *before* design changes are finalized, allowing us to proactively update our tests. I attend design syncs when necessary, adapt to their working conventions, and ensure that design tickets are always explicitly linked to corresponding QA tasks.
- **prevent misunderstandings regarding visual changes;** Clear, shared acceptance criteria on every ticket are mandatory. I foster a culture of open communication, encouraging the QA team to ask clarifying questions immediately if a visual detail or intended behavior is ambiguous.

Quality must be a shared mindset; therefore, passing component tests (like storybook tests) should be a standard acceptance criterion for both design and development tickets.

4. Challenging Situation

Describe one difficult situation you faced in QA or test automation. Include:

- **what happened;** I have managed high-stakes automation projects across critical domains like fintech and medical. Two notable examples highlight different types of challenges:

Design Case (HR Management Application): Validating a product with constantly updating designs and content across 6 languages and 2 environments (UAT and Production). The difficulty lay in reliably detecting broken links, UI overlap, and rendering issues without creating brittle tests.

Complexity Case (Custom Firmware for Earbuds): Automating tests for custom earbud firmware and its interaction with a companion mobile app. This required flashing firmware to physical devices via USB (across Windows COM and Linux tty), initiating serial connections, managing remote connections to a custom device farm in Munich, validating audio I/O (wake words), and automating Slack/Email reporting.

- **what decision you made;**

Design Case Decisions: I implemented a highly modular, Playwright/TypeScript framework capable of handling all languages and environments dynamically. To eliminate flakiness from routine content changes, I coordinated with development to create a dedicated visual-testing environment with static mock content, enabling stable component and E2E visual regression tests.

Complexity Case Decisions: I engineered a highly portable framework that programmatically downloaded releases via S3 APIs and handled OS-level hardware interactions (serial, ADB, SSH). I integrated Google Chirp for speech-to-text audio validation, managed parallel execution across a device pool, and implemented an automated failure analysis system using LLMs to parse stack traces and images, with self-assessment and entropy calculations used for outputting the LLM analysis accuracy.

- **how you communicated with the team and stakeholders;** I prioritize direct, friendly, and transparent communication. For both cases, I documented the exact bottlenecks, proposed solutions, and outlined required support. Set up targeted syncs only with the relevant stakeholders. I ensured every meeting resulted in clear action items and maintained high visibility on the project's status throughout its lifecycle.

- **what you learned from the experience.** I learned that proactive planning and highly efficient, documented communication are the lifelines of successful remote engineering collaborations.